```
import os
import pandas as pd
import numpy as np
file_path = "/content/drive/MyDrive/Mobifall/MobiFall_All.csv"  # change t
```

```
df = pd.read_csv(file_path)

print("Data Loaded Successfully")
```

```
Data Loaded Successfully
```

```
df.head()
print(df.columns)
df.info()
```

```
Index(['timestamp', 'x', 'y', 'z', 'activity_code', 'sensor_type',
       'subject_id', 'trial_number', 'category', 'azimuth', 'pitch', 'roll'],
      dtype='object')
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5804826 entries, 0 to 5804825
Data columns (total 12 columns):
 #   Column         Dtype
---  ------         -----
 0   timestamp      int64
 1   x              float64
 2   y              float64
 3   z              float64
 4   activity_code  object
 5   sensor_type    object
 6   subject_id     int64
 7   trial_number   int64
 8   category       object
 9   azimuth        float64
 10  pitch          float64
 11  roll           float64
dtypes: float64(6), int64(3), object(3)
memory usage: 531.4+ MB
```

```
# Convert category to binary label
df['label'] = df['category'].apply(lambda x: 1 if x == 'FALL' else 0)

print(df['label'].value_counts())
```

```
label
0    4416485
1    1388341
Name: count, dtype: int64
```

```
# Separate sensors
df_acc = df[df['sensor_type'] == 'acc'].copy()
df_gyro = df[df['sensor_type'] == 'gyro'].copy()
```

```
# Sort to maintain sequence order
df_acc = df_acc.sort_values(['subject_id', 'trial_number', 'timestamp'])
df_gyro = df_gyro.sort_values(['subject_id', 'trial_number', 'timestamp'])
```

```
def create_windows(df, features, window_size=50, stride=10):
    Xs, ys = [], []

    # Group by subject & trial (VERY IMPORTANT)
    grouped = df.groupby(['subject_id', 'trial_number'])

    for _, group in grouped:
        data = group[features].values
        labels = group['label'].values

        # Sliding window
        for i in range(0, len(data) - window_size, stride):
            Xs.append(data[i:i+window_size])

            # Majority label inside window
            ys.append(int(np.round(np.mean(labels[i:i+window_size]))))

    return np.array(Xs), np.array(ys)
```

```
# Accelerometer (x,y,z)
X_acc, y_acc = create_windows(df_acc, ['x', 'y', 'z'])

# Gyroscope (x,y,z)
X_gyro, y_gyro = create_windows(df_gyro, ['x', 'y', 'z'])

print("ACC shape:", X_acc.shape)
print("GYRO shape:", X_gyro.shape)
```

```
ACC shape: (104028, 50, 3)
GYRO shape: (238503, 50, 3)
```

```
from sklearn.preprocessing import MinMaxScaler

def normalize_data(X):
    scaler = MinMaxScaler()

    # reshape → scale → reshape back
    X_reshaped = X.reshape(-1, X.shape[2])
    X_scaled = scaler.fit_transform(X_reshaped)

    return X_scaled.reshape(X.shape), scaler

X_acc, scaler_acc = normalize_data(X_acc)
X_gyro, scaler_gyro = normalize_data(X_gyro)
```

```
from sklearn.model_selection import train_test_split
```

```
X_acc_train, X_acc_test, y_acc_train, y_acc_test = train_test_split(
    X_acc, y_acc, test_size=0.2, random_state=42
)

X_gyro_train, X_gyro_test, y_gyro_train, y_gyro_test = train_test_split(
    X_gyro, y_gyro, test_size=0.2, random_state=42
)
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

def build_model(input_shape):
    model = Sequential([
        LSTM(64, input_shape=input_shape),
        Dense(32, activation='relu'),
        Dense(1, activation='sigmoid')
    ])

    model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    return model
```

```
# Accelerometer model
model_acc = build_model((50, 3))
model_acc.fit(X_acc_train, y_acc_train, epochs=5, validation_data=(X_acc_tes

# Gyroscope model
model_gyro = build_model((50, 3))
model_gyro.fit(X_gyro_train, y_gyro_train, epochs=5, validation_data=(X_gyro
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: User
  super().__init__(**kwargs)
Epoch 1/5
2601/2601 ━━━━━━━━━━━━━━━━━━━━ 64s 24ms/step - accuracy: 0.9138 - loss: 0.240
Epoch 2/5
2601/2601 ━━━━━━━━━━━━━━━━━━━━ 66s 25ms/step - accuracy: 0.9267 - loss: 0.197
Epoch 3/5
2601/2601 ━━━━━━━━━━━━━━━━━━━━ 62s 24ms/step - accuracy: 0.9308 - loss: 0.180
Epoch 4/5
2601/2601 ━━━━━━━━━━━━━━━━━━━━ 82s 24ms/step - accuracy: 0.9329 - loss: 0.173
Epoch 5/5
2601/2601 ━━━━━━━━━━━━━━━━━━━━ 69s 27ms/step - accuracy: 0.9335 - loss: 0.171
Epoch 1/5
5963/5963 ━━━━━━━━━━━━━━━━━━━━ 148s 25ms/step - accuracy: 0.7604 - loss: 0.55
Epoch 2/5
5963/5963 ━━━━━━━━━━━━━━━━━━━━ 202s 25ms/step - accuracy: 0.7604 - loss: 0.55
Epoch 3/5
5963/5963 ━━━━━━━━━━━━━━━━━━━━ 154s 26ms/step - accuracy: 0.7604 - loss: 0.55
Epoch 4/5
5963/5963 ━━━━━━━━━━━━━━━━━━━━ 200s 25ms/step - accuracy: 0.7604 - loss: 0.55
Epoch 5/5
```

```
5963/5963 ━━━━━━━━━━━━━━━━━━━━ 144s 24ms/step - accuracy: 0.7604 - loss: 0.55
<keras.src.callbacks.history.History at 0x78a48b9dca40>
```

```
print("ACC Model Accuracy:")
print(model_acc.evaluate(X_acc_test, y_acc_test))

print("GYRO Model Accuracy:")
print(model_gyro.evaluate(X_gyro_test, y_gyro_test))
```

```
ACC Model Accuracy:
651/651 ━━━━━━━━━━━━━━━━━━━━ 5s 8ms/step - accuracy: 0.9332 - loss: 0.1651
[0.16506266593933105, 0.9331923723220825]
GYRO Model Accuracy:
1491/1491 ━━━━━━━━━━━━━━━━━━━━ 13s 9ms/step - accuracy: 0.7614 - loss: 0.5491
[0.549050509929657, 0.761430561542511]
```

```
def fusion_predict(sample_acc, sample_gyro):
    prob_acc = model_acc.predict(sample_acc)[0][0]
    prob_gyro = model_gyro.predict(sample_gyro)[0][0]

    # Average fusion
    final_prob = (prob_acc + prob_gyro) / 2

    return final_prob

# Test on few samples
for i in range(5):
    acc_sample = X_acc_test[i].reshape(1, 50, 3)
    gyro_sample = X_gyro_test[i].reshape(1, 50, 3)

    print("Final Prob:", fusion_predict(acc_sample, gyro_sample))
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 296ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 281ms/step
Final Prob: 0.12323725
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 59ms/step
Final Prob: 0.12505001
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
Final Prob: 0.12387295
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
Final Prob: 0.11254812
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
Final Prob: 0.12411575
```

```
def test_single_sample(index):
    acc_sample = X_acc_test[index].reshape(1, 50, 3)
    gyro_sample = X_gyro_test[index].reshape(1, 50, 3)

    prob_acc = model_acc.predict(acc_sample)[0][0]
    prob_gyro = model_gyro.predict(gyro_sample)[0][0]
```

```
    final_prob = (prob_acc + prob_gyro) / 2

    print(f"Index: {index}")
    print(f"Actual Label: {y_acc_test[index]}")
    print(f"Acc Prob: {prob_acc:.4f}")
    print(f"Gyro Prob: {prob_gyro:.4f}")
    print(f"Final Prob: {final_prob:.4f}")
    if(final_prob>0.5):
      print("Final result: Fall")
    else:
      print("Final result: ADL")
    print("-" * 40)
```

```
for i in range(10):
    test_single_sample(i)
```

```
Gyro Prob: 0.2228
Final Prob: 0.1125
Final result: ADL
----------------------------------------
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
Index: 4
Actual Label: 0
Acc Prob: 0.0098
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
Index: 8
Actual Label: 1
Acc Prob: 0.9887
Gyro Prob: 0.2384
Final Prob: 0.6136
Final result: Fall
----------------------------------------
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
Index: 9
Actual Label: 0
Acc Prob: 0.0042
Gyro Prob: 0.2384
Final Prob: 0.1213
Final result: ADL
----------------------------------------
```

Start coding or generate with AI.

```
#edge case: When all are zeroes

dummy = np.zeros((1, 50, 3))

print("Acc:", model_acc.predict(dummy))
print("Gyro:", model_gyro.predict(dummy))
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
Acc: [[0.9982914]]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
Gyro: [[0.23643574]]
```

```
THRESHOLD = 0.4

def detect_fall(acc_sample, gyro_sample):
    """
    acc_sample  : shape (50, 3)
    gyro_sample : shape (50, 3)
    """

    # reshape for model
    acc_sample = acc_sample.reshape(1, 50, 3)
    gyro_sample = gyro_sample.reshape(1, 50, 3)

    # model predictions
    prob_acc = model_acc.predict(acc_sample)[0][0]
    prob_gyro = model_gyro.predict(gyro_sample)[0][0]

    # 🔥 Fusion (slightly biased towards detecting fall)
    final_prob = (0.6 * prob_acc) + (0.4 * prob_gyro)

    # decision
    if final_prob > THRESHOLD:
        return "FALL", final_prob
```

```
    else:
        return "ADL", final_prob
```

```
# test few samples
for i in range(10):
    result, prob = detect_fall(X_acc_test[i], X_gyro_test[i])

    print(f"Actual: {y_acc_test[i]} | Predicted: {result} | Prob: {prob:.3f}
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
Actual: 0 | Predicted: ADL | Prob: 0.100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step
Actual: 0 | Predicted: ADL | Prob: 0.102
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step
Actual: 0 | Predicted: ADL | Prob: 0.101
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
Actual: 0 | Predicted: ADL | Prob: 0.090
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
Actual: 0 | Predicted: ADL | Prob: 0.101
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
Actual: 1 | Predicted: ADL | Prob: 0.290
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
Actual: 0 | Predicted: ADL | Prob: 0.101
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
Actual: 0 | Predicted: ADL | Prob: 0.099
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
Actual: 1 | Predicted: FALL | Prob: 0.689
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
Actual: 0 | Predicted: ADL | Prob: 0.098
```

```
fall_indices = [i for i in range(len(y_acc_test)) if y_acc_test[i] == 1]

for i in fall_indices[:20]:
    result, prob = detect_fall(X_acc_test[i], X_gyro_test[i])

    print(f"Actual: FALL | Predicted: {result} | Prob: {prob:.3f}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 54ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step
Actual: FALL | Predicted: ADL | Prob: 0.290
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 58ms/step
Actual: FALL | Predicted: FALL | Prob: 0.689
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step
Actual: FALL | Predicted: FALL | Prob: 0.522
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step
Actual: FALL | Predicted: FALL | Prob: 0.694
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 58ms/step
Actual: FALL | Predicted: FALL | Prob: 0.695
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 54ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step
Actual: FALL | Predicted: FALL | Prob: 0.568
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 68ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 71ms/step
Actual: FALL | Predicted: FALL | Prob: 0.519
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 65ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step
Actual: FALL | Predicted: ADL | Prob: 0.207
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
Actual: FALL | Predicted: FALL | Prob: 0.693
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
Actual: FALL | Predicted: FALL | Prob: 0.510
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
Actual: FALL | Predicted: FALL | Prob: 0.577
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
Actual: FALL | Predicted: ADL | Prob: 0.365
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
Actual: FALL | Predicted: FALL | Prob: 0.509
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
Actual: FALL | Predicted: FALL | Prob: 0.595
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
Actual: FALL | Predicted: ADL | Prob: 0.368
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
Actual: FALL | Predicted: FALL | Prob: 0.695
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
Actual: FALL | Predicted: FALL | Prob: 0.660
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
Actual: FALL | Predicted: FALL | Prob: 0.667
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
Actual: FALL | Predicted: FALL | Prob: 0.454
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step
```

```
normal_indices = [i for i in range(len(y_acc_test)) if y_acc_test[i] == 0]

for i in normal_indices[:5]:
    result, prob = detect_fall(X_acc_test[i], X_gyro_test[i])

    print(f"Actual: ADL | Predicted: {result} | Prob: {prob:.3f}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
Actual: ADL | Predicted: ADL | Prob: 0.100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step
Actual: ADL | Predicted: ADL | Prob: 0.102
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 68ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step
Actual: ADL | Predicted: ADL | Prob: 0.101
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 58ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 57ms/step
Actual: ADL | Predicted: ADL | Prob: 0.090
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 59ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step
Actual: ADL | Predicted: ADL | Prob: 0.101
```

Start coding or generate with AI.

```
model_acc.save("acc_model.h5")
model_gyro.save("gyro_model.h5")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
```

```
import tensorflow as tf

# Convert ACC model
converter = tf.lite.TFLiteConverter.from_keras_model(model_acc)
# Apply the suggested fix for dynamic shapes in LSTM models
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS, tf.li
converter._experimental_lower_tensor_list_ops = False
tflite_acc = converter.convert()

with open("acc_model.tflite", "wb") as f:
    f.write(tflite_acc)

# Convert GYRO model
converter = tf.lite.TFLiteConverter.from_keras_model(model_gyro)
# Apply the suggested fix for dynamic shapes in LSTM models
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS, tf.li
converter._experimental_lower_tensor_list_ops = False
tflite_gyro = converter.convert()

with open("gyro_model.tflite", "wb") as f:
    f.write(tflite_gyro)

print("TFLite models ready")
```

```
Saved artifact at '/tmp/tmpg7tk90gw'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 50, 3), dtype=tf.float32,
Output Type:
  TensorSpec(shape=(None, 1), dtype=tf.float32, name=None)
Captures:
  132648426321296: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132648426323216: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132648426322640: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
  132648426323600: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132648426320144: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132648426323408: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132648426321104: TensorSpec(shape=(), dtype=tf.resource, name=None)
Saved artifact at '/tmp/tmps8le7zs6'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 50, 3), dtype=tf.float32,
Output Type:
  TensorSpec(shape=(None, 1), dtype=tf.float32, name=None)
Captures:
  132648112052688: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132648112052304: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132648112047120: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132648112047504: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132648112050576: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132648112050768: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132648112048080: TensorSpec(shape=(), dtype=tf.resource, name=None)
TFLite models ready
```

4. FINAL INPUT FORMAT FOR APP Accelerometer: { "data": [[x, y, z], ... 50 rows] } Gyroscope: { "data": [[x, y, z], ... 50 rows] } 📤 15. FINAL LOGIC IN APP prob_acc = model_acc(input) prob_gyro = model_gyro(input)

final_prob = (prob_acc + prob_gyro) / 2

if final_prob > 0.7: FALL 🔥 16. WHAT YOU ACHIEVED

✓ No sensor sync issue ✓ Real-time ready ✓ Mobile deployable ✓ Scalable design ✓ Viva strong

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```